EXP.NO: 8	Develop an application to change the font and color of the text and display toast message when the user presses the button.

AIM:

To develop an Android application that allows the user to change the font and color of text displayed on the screen and display a Toast message when a button is pressed.

ALGORITHM:

1. **Initialize the Android Application:**
   - Set up a new Android project in **Android Studio**.
   - Create a **MainActivity** Java file for the main screen of the application.
2. **Design the User Interface (UI):**
   - In the **XML layout file**, add the following UI components:
     - **TextView**: For displaying the text whose font and color will be changed.
     - **Button**: To trigger the change in font and color, and show a **Toast message**.
     - **Spinner or Color Picker**: To let the user choose a color (optional, or a simple button for a predefined color change).
     - **RadioButtons** or a **Dropdown Menu** to select different font styles (optional).
3. **Declare Variables:**
   - Create references for the **TextView**, **Button**, **Color Picker** (if used), and **Radio Buttons** (if used to change font style).
   - Define variables for the **font style**, **color**, and the **message** to display in the **Toast**.
4. **Set Event Listener for the Button:**
   - Set an **OnClickListener** for the **Button**.
   - Inside the onClick() method, write the logic to:
     - Change the **font** of the **TextView** based on the selected font style.

        - Change the **color** of the **TextView** based on the selected color.
        - Display a **Toast message** indicating the action has been performed.

5. **Implement Font Style Changes:**
    - Change the **TextView's font** using setTypeface() to apply a selected font style (e.g., bold, italic).
    - You can either use predefined styles (like Typeface.BOLD or Typeface.ITALIC) or let the user choose from a list of font styles.
6. **Implement Color Change:**
    - Change the **TextView's text color** by using the setTextColor() method and applying the selected color (e.g., Color.RED, Color.BLUE, or a color from the **Color Picker**).
    - If using a color picker or dropdown, get the selected color and set it on the **TextView**.
7. **Show Toast Message:**
    - Display a **Toast** message to the user when the button is clicked, informing them that the text style or color has changed. You can also show a message like "Font and color changed!" or any other appropriate message.
8. **End the Operation:**
    - After the changes are made, the application will display the updated **TextView** with the new font and color, and the **Toast message** will appear to provide feedback.

Code:

Step 1: Create the Project

- **Template**: Empty Views Activity
- **Project name** – Font_Change
- **Package** - com.example.font_change
- **Language**: Java
- **Minimum SDK** – API 24
- **Build system**: Groovy DSL

Step 2: Under res → layout → Updated activity_main.xml using **ConstraintLayout**

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/mainLayout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <TextView
        android:id="@+id/sampleText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, World!!!!!!"
        android:textSize="24sp"
        android:textColor="#000000"
        android:fontFamily="@font/roboto_regular"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toTopOf="@+id/changeStyleButton"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintVertical_bias="0.4"/>

    <Button
        android:id="@+id/changeStyleButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Change Font Style"
        app:layout_constraintTop_toBottomOf="@id/sampleText"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_marginTop="20dp"/>
</androidx.constraintlayout.widget.ConstraintLayout>
```

Step 3: Add a Custom Font in Android Studio

Add the Font Resource

1. In Android Studio:

- Right-click on res/ → **New** → **Android Resource Directory**

2. In the dialog:
   - **Directory name**: font
   - **Resource type**: Select font from the dropdown
   - Click **OK**
3. Download the font:
   - Open the link → https://fonts.google.com/
   - Select the font Roboto → Get Font → Download all
   - Open the downloaded zip file and go to the folder "static". Choose the font "Roboto-Regular.ttf" and rename to "roboto-regular.ttf". copy and paste this inside the font folder under drawable in Android studio.

Step 3 : Update the MainActivity.java file with this content

```
package com.example.font_change;


import android.graphics.Color;
import android.graphics.Typeface;
import android.os.Bundle;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.res.ResourcesCompat;

public class MainActivity extends AppCompatActivity {

    TextView sampleText;
    Button changeStyleButton;
    boolean isChanged = false;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        sampleText = findViewById(R.id.sampleText);
        changeStyleButton = findViewById(R.id.changeStyleButton);
```

```
        changeStyleButton.setOnClickListener(v -> {
            if (!isChanged) {
                sampleText.setTextColor(Color.parseColor("#FF5722")); // Orange
                Typeface typeface = ResourcesCompat.getFont(this, R.font.roboto_regular);
                sampleText.setTypeface(getResources().getFont(R.font.roboto_regular),
Typeface.BOLD_ITALIC);
                Toast.makeText(MainActivity.this, "Style Changed!",
Toast.LENGTH_SHORT).show();
            } else {
                sampleText.setTextColor(Color.BLACK);
                sampleText.setTypeface(Typeface.DEFAULT);
                Toast.makeText(MainActivity.this, "Style Reset!",
Toast.LENGTH_SHORT).show();
            }
            isChanged = !isChanged;
        });
    }
}
```

Step 4 – Under Gradle Scripts section → update the build.gradle(Module:app)

```
plugins {
    alias(libs.plugins.androidApplication)
}

android {
    namespace 'com.example.font_change'
    compileSdk 34

    defaultConfig {
        applicationId "com.example.font_change"
        minSdk 26
        targetSdk 34
        versionCode 1
        versionName "1.0"

        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-
```

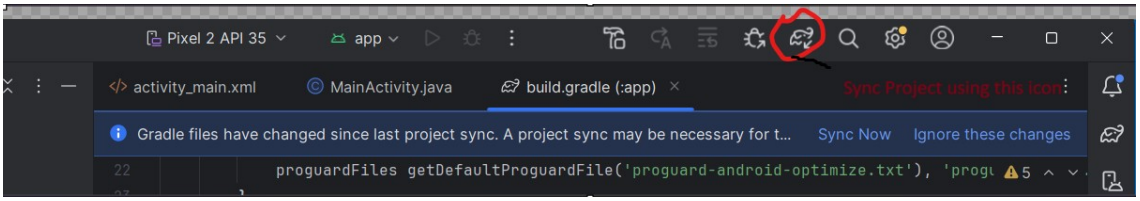
```
rules.pro'
        }
    }

    compileOptions {
        sourceCompatibility JavaVersion.VERSION_1_8
        targetCompatibility JavaVersion.VERSION_1_8
    }
}

dependencies {
    // View/Layout
    implementation 'androidx.constraintlayout:constraintlayout:2.1.4'

    // Core libraries
    implementation 'androidx.core:core:1.12.0'
    implementation libs.appcompat
    implementation libs.material
    implementation libs.activity

    // Testing
    testImplementation libs.junit
    androidTestImplementation libs.ext.junit
    androidTestImplementation libs.espresso.core
}
```



Step 5 – Click the Play Button to Run the project using the Emulator and verify the output as below

Output:

RESULT:

Thus, An mobile application was developed in Android studio to change the font and color of the text and display toast message when the user presses the button.